From Findings to Verifiable Remediation Artifacts: A Smart Contract Security Pipeline

Fernando Boiero
Universidad Tecnológica Nacional (UTN)
Facultad Regional Villa María
Villa María, Córdoba, Argentina
fboiero@frvm.utn.edu.ar

Sebastian Norberto Mussetta
Universidad Tecnológica Nacional (UTN)
Facultad Regional Villa María
Villa María, Córdoba, Argentina
smussetta@frvm.utn.edu.ar

***Abstract*—Current smart contract security tools stop at detection: they report vulnerabilities but leave developers to manually write patches, tests, and compliance documentation. We present MIESC's automated remediation pipeline, which extends vulnerability detection into a complete find–fix–verify workflow. Given a Solidity contract, the pipeline: (1) detects vulnerabilities using the MIESC detection stack (93.7% recall on the latest SmartBugs full-corpus reproducible profile, 92.5% recall on an EVMBench local high-severity extraction with a multi-provider ensemble), (2) generates self-contained patch candidates (141/143 fix application, 90/141 standalone compilation, 93/141 vulnerability elimination by re-scan, and 91/141 bounded no-regression), (3) produces Foundry exploit tests that confirm supported vulnerabilities on the original contract and verify the patched path, (4) generates formal verification specifications (CVL, Scribble, SMTChecker), and (5) maps findings to 12 international security standards for compliance reporting. We evaluate remediation on the SmartBugs-curated corpus (143 contracts, 10 vulnerability categories) and trace the complete pipeline on an illustrative vulnerable contract. Among five compared repair tools, MIESC is the only evaluated tool that generates exploit tests, formal specifications, and compliance documentation alongside patches. The pipeline can run locally without API calls. MIESC is available under AGPL-3.0.**

***Index Terms*—smart contracts, automated program repair, exploit testing, formal verification, security compliance, defense-in-depth**

I. Introduction

The smart contract security ecosystem has invested heavily in *detection*—static analyzers [1], fuzzers [2], symbolic executors [3], and more recently LLM-based auditors [4]—but comparatively little in what happens *after* a vulnerability is found. In practice, the workflow looks like this:

1) A tool reports: "Reentrancy in `withdraw()` (HIGH)."
2) A developer manually writes a patch (add `nonReentrant`).
3) The developer manually writes a test checking the intended fix path.
4) The developer manually maps the finding to compliance requirements.
5) The developer manually writes the audit report.

Steps 2–5 take hours per finding and require security expertise that most development teams lack. This paper presents MIESC's automated remediation pipeline, which exposes the workflow through composable commands:

```
miesc scan contract.sol -o results.json
miesc fix results.json \
  -c contract.sol -o fixed.sol
miesc test-gen results.json -c contract.sol
miesc compliance results.json --standard mica
miesc report results.json -t premium -f pdf
```

We evaluate patch quality on the SmartBugs-curated corpus (143 contracts, 10 vulnerability categories), trace the complete artifact pipeline on an illustrative vulnerable contract, and compare against five prior automated repair tools. Our contributions are:

1) **Self-contained patch generation** (`miesc fix`): text-level Solidity patch candidates that avoid adding external dependencies. Evaluated on 143 contracts: 99% fix application rate, 64% standalone compilation, 66% vulnerability elimination, and 65% pass rate under a bounded re-scan no-regression criterion.
2) **Exploit test generation** (`miesc test-gen`): Foundry test templates that demonstrate exploitability on the vulnerable contract and verify that the patched version blocks the same exploit path.
3) **Formal specification generation** (`miesc specs`): auto-generated Certora CVL rules, Scribble annotations, and SMTChecker assertions from findings.
4) **Compliance mapping** (`miesc compliance`): automated mapping to 12 international standards (ISO 27001, NIST CSF, OWASP, MiCA, DORA, CWE, SWC).
5) **Corpus-scale evaluation with comparison**: systematic evaluation against SCRepair, sGuard, Elysium, ACFIX, and SCPatcher, showing that MIESC covers a broader remediation artifact set than prior repair-only tools.

II. Related Work

A. Automated Program Repair for Smart Contracts

SCRepair [5] uses genetic programming to mutate Solidity source code until tests pass, but requires pre-existing test suites. sGuard [6] inserts runtime checks (reentrancy locks, integer overflow guards) at near-100% compilation but does not generate tests to verify the fix. Elysium [7] operates at

the bytecode level, achieving +30% fix rate over its predecessor Shield but limited to 7 SWC categories. ACFIX [8] uses LLMs guided by mined RBAC practices to fix access control vulnerabilities, achieving 94.9% fix rate but targeting only a single vulnerability class. SCPatcher [9] applies RAG-enhanced LLMs to general-purpose patching, reporting 81.5% fix rate and 91% compilation.

MIESC differs in three ways: (1) patch candidates are *self-contained* in the sense that they avoid new library imports, (2) exploit tests are generated alongside patches to check the vulnerable and patched paths, and (3) the pipeline extends beyond patching to formal specifications and compliance mapping. Section IV-G provides a detailed feature comparison.

B. Test Generation for Smart Contracts

Foundry [10] and Echidna [2] provide fuzzing and property-based testing frameworks, but require developers to write test properties manually. Harvey [11] generates high-coverage test inputs but not exploit-specific test cases.

MIESC's `test-gen` command generates vulnerability-specific exploit tests: a reentrancy finding produces a test with an attacker contract that re-enters the victim, while a selfdestruct finding produces an unauthorized-caller test. The generated artifacts serve two roles: exploit-confirmation tests demonstrate the vulnerable behavior on the original contract, and fixed-version regression tests verify that the same path is blocked after patching.

C. Compliance Automation

In the repair tools surveyed in this paper, compliance mapping is outside the reported scope. Mapping findings to standards such as ISO 27001, MiCA Art. 11, and NIST CSF is currently a manual process performed by auditors during report writing. MIESC automates this with a rule-based mapper covering 12 standards.

D. LLM-Assisted Security Analysis

GPTScan [4] combines GPT-4 with program analysis for logic vulnerability detection but produces no patches. EVM-Bench [12] evaluates LLM agents on real audit reports but measures only detection recall, not remediation. The SmartBugs framework [13] provides execution infrastructure for tool evaluation but includes no patching capabilities. MIESC builds on these detection advances by extending the pipeline beyond detection into actionable remediation artifacts.

III. Pipeline Architecture

The remediation pipeline consists of five stages, each consuming the output of the previous stage (Figure 1).

A. Stage 1: Detection

Described in our companion paper [14]. The latest SmartBugs full-corpus reproducible profile reaches 93.7% recall on 143 contracts, while the EVMBench local high-severity extraction reaches 92.5% recall using a four-provider union ensemble over 120 local findings. The scanner outputs a normalized JSON with findings, severity, location, confidence, canonical category, and remediation fields consumed by the patcher.

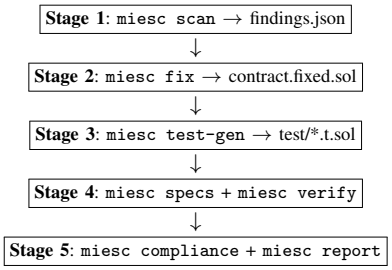

Fig. 1. MIESC remediation pipeline. Each stage consumes the output of the previous stage, producing a progressively more complete security artifact.

B. Stage 2: Patch Generation

`miesc fix` applies text-level patches to the original Solidity source. The patcher operates on the finding's type and function location:

- **Reentrancy**: inserts `nonReentrant` modifier. If OpenZeppelin is not detected, inlines a self-contained `MiescReentrancyGuard` abstract contract (11 lines).
- **Access control / selfdestruct**: inserts `onlyOwner` modifier. If no modifier is defined, inlines a 5-line `onlyOwner` modifier using the existing `owner` state variable.
- **Unchecked calls**: wraps value-bearing low-level calls in `require(success)`.
- **Arithmetic (pre-0.8)**: inserts `using SafeMath for uint256`.

A deduplication step tracks `(function, fix_category)` pairs to avoid applying the same fix twice when multiple findings target the same function (e.g., `suicidal` and `selfdestruct-identifier` both targeting `destroy()`).

Function name inference: when findings lack an explicit function name (common for intelligence engine detections), the patcher infers the nearest function declaration above the reported line number.

1) Patch Example: Reentrancy Guard Injection: Listing 1 shows a vulnerable withdrawal function. The external call on line 3 occurs *before* the state update on line 4, enabling a classic reentrancy attack. Listing 2 shows MIESC's patched version: the inline `MiescReentrancyGuard` (lines 1–11) provides a self-contained mutex that compiles without external dependencies, while the `nonReentrant` modifier on line 13 blocks re-entry. This inline approach avoids requiring OpenZeppelin imports, which is critical for standalone compilation of legacy contracts.

Listing 1. Vulnerable: external call before state update

```
1 function withdraw(uint256 amount) external {
2     require(balances[msg.sender] >= amount);
3     (bool ok,) = msg.sender.call{value: amount}("");
4     require(ok);
5     balances[msg.sender] -= amount;
6 }
```

The inline guard is compatible with Solidity $\geq$0.4.22 (the minimum version supporting `constructor` syntax).

Listing 2. Patched: inline reentrancy guard injected by MIESC

```
contract MiescReentrancyGuard {
    bool internal _locked;
    modifier nonReentrant() {
        require(!_locked, "ReentrancyGuard: reentrant");
        _locked = true;
        _;
        _locked = false;
    }
}

contract Vault is MiescReentrancyGuard {
    // ...
    function withdraw(uint256 amount)
        external nonReentrant
    {
        require(balances[msg.sender] >= amount);
        (bool ok,) = msg.sender.call{value: amount}("");
        require(ok);
        balances[msg.sender] -= amount;
    }
}
```

For contracts already inheriting from OpenZeppelin's `ReentrancyGuard`, the patcher detects the existing import and adds only the modifier annotation.

C. Stage 3: Exploit Test Generation

`miesc test-gen` generates one Foundry test file per finding. Each test is designed with a dual-assertion property:

- **Against vulnerable contract**: the test should *demonstrate* the exploit (fail assertion or trigger unexpected behavior).
- **Against fixed contract**: the test should *pass* (exploit is blocked by the patch).

Templates exist for reentrancy (deploys attacker contract with re-entrant `receive()`), access control (`vm.prank` + `vm.expectRevert`), selfdestruct (verifies non-owner call reverts), and unchecked calls (reverting receiver contract).

D. Stage 4: Formal Specification

`miesc specs` generates executable specifications from findings:

- **Certora CVL**: a `noReentrancy` rule targeting the vulnerable function.
- **Scribble**: inline annotations (`/// #if_succeeds ...`).
- **SMTChecker**: `require(success)` assertions for unchecked calls.

`miesc verify` executes specifications against available provers (Certora, Halmos, solc SMTChecker).

E. Stage 5: Compliance and Reporting

`miesc compliance` maps each finding to applicable standards using a rule-based mapper with 12 standard profiles. `miesc report` generates audit reports in HTML, PDF, Markdown, or SARIF format with CVSS scores, remediation roadmaps, and executive summaries.

IV. Evaluation

A. Experimental Setup

We evaluate the remediation stage on the SmartBugs-curated corpus [15]: 143 contracts spanning 10 vulnerability categories (reentrancy, access control, unchecked low-level calls, arithmetic, denial of service, front running, short addresses, time manipulation, bad randomness, and other). The remediation metrics are generated by `benchmarks/fix_eval.py` and stored in `benchmarks/results/fix_eval_results.json`; Paper 2 claims are traced in `paper2_claims_matrix.json`. Derived artifacts further summarize patch quality by transformation class and standalone compilation failures by category.

Metric definitions. *Fix applied* means the patcher produced a modified source file for at least one fixable HIGH/CRITICAL finding. *Standalone compilation* means the patched file compiles without restoring the original project dependency tree. *Vulnerability eliminated* means a re-scan reports fewer HIGH/CRITICAL findings for the target category. *No regression* is a bounded re-scan criterion: the patched contract does not materially increase the finding count under the same scanner; it is not a proof of full functional equivalence.

As an illustrative example, we also trace the full pipeline end-to-end on a single test contract (`AuditTarget.sol`) containing 5 intentional vulnerabilities (reentrancy, missing access control, unprotected selfdestruct, missing zero-address check, and centralized emergency withdrawal).

Environment: macOS ARM64 (Apple Silicon), 48GB RAM, Foundry v1.3.5, Solidity 0.8.20. The Paper 2 remediation evaluation was re-run after the final Paper 1 detection profile; Paper 2 treats remediation metrics and Paper 1 detection metrics as separate reproducibility artifacts.

B. Stage 2: Patch Quality

TABLE I
AGGREGATE PATCH GENERATION RESULTS (143 CONTRACTS)

Metric	Count	Rate
Fix applied	141/143	99%
Compilation (standalone)	90/141	64%
Vulnerability eliminated	93/141	66%
No-regression criterion	91/141	65%
Fix failed	0/141	0%

Table I shows aggregate results across the 143-contract corpus. Of the 143 contracts, 141 receive a successfully applied patch (99%). Of those 141, 90 compile standalone without external dependencies (64%), 93 reduce HIGH/CRITICAL findings for the target category upon re-scan (66%), and 91 satisfy the bounded no-regression criterion (65%).

Table II breaks down results by category. Reentrancy is the strongest transformation: 30/31 patched contracts compile standalone and 30/31 reduce the target finding on re-scan. Arithmetic also compiles reliably (15/15), but the current SafeMath-style mitigation eliminates only 1/15 target findings under the scanner, indicating that simple arithmetic wrapping is insufficient for many historical SmartBugs patterns.

TABLE II
PER-CATEGORY PATCH RESULTS

Category	Applied	Compile	Eliminated	No Regr.
Reentrancy	31	30 (97%)	30 (97%)	22 (71%)
Access Control	17	11 (65%)	9 (53%)	8 (47%)
Unchecked Calls	51	20 (39%)	38 (75%)	42 (82%)
Arithmetic	15	15 (100%)	1 (7%)	2 (13%)
Others (6 cats)	27	14 (52%)	15 (56%)	17 (63%)
Total	**141**	**90 (64%)**	**93 (66%)**	**91 (65%)**

Unchecked low-level calls remain the main compilation bottleneck: `require(success)` reduces the target finding in 38/51 patched contracts, but only 20/51 compile as standalone files.

Viewed by transformation class, reentrancy guard injection applies to 31 contracts, compiles in 30, and eliminates the target finding in 30. The `require(success)` wrapper applies to 51 unchecked-call contracts, eliminates 38 target findings, and accounts for 31 of the 51 standalone compilation failures. This indicates that the transformation often reduces the scanner-visible vulnerability, while surrounding legacy contracts still fail standalone compilation because of unresolved symbols or existing source-level irregularities.

Experiment validity audit. To separate patch quality from corpus compilability, we additionally compile the original contracts under the same standalone criterion. Among the 141 patched contracts, all 141 original files compile standalone, while 90 patched files compile (64%). Joint success is stricter than any single metric: 42/141 patches both compile and reduce the target finding, and 30/141 compile, reduce the target finding, and satisfy the bounded no-regression criterion. No-regression is threshold-sensitive: under a zero-new-finding threshold 64/141 pass, under the reported +2 threshold 91/141 pass, and under a +5 threshold 139/141 pass.

C. Stage 3: Test Validity

TABLE III
GENERATED FOUNDRY TEST RESULTS

Test	Compile	Vuln.	Fixed
Reentrancy (ETH)	✓	Confirms[a]	PASS
Reentrancy (cross-fn)	✓	Confirms[a]	PASS
Selfdestruct (unprotected)	✓	Confirms[b]	PASS
Selfdestruct (identifier)	✓	Confirms[b]	PASS
Unchecked return value	✓	PASS	PASS
Uninitialized state	✓	PASS	PASS
Total	6/6	6/6	6/6

[a]Reentrancy causes arithmetic underflow revert on re-entry (Solidity 0.8+ checked arithmetic). [b]Selfdestruct tests expect `vm.expectRevert` but call succeeds on vulnerable contract, evidencing missing access control.

All 6 generated tests in the illustrative end-to-end case compile with Foundry. Against the vulnerable contract, each test demonstrates the intended vulnerability condition. Against the fixed contract, all 6 tests pass, indicating that the generated patches block the exercised exploit paths.

Exploit drain demonstration. Listing 3 shows the Foundry test generated by `miesc test-gen` for the reentrancy finding. The test deploys an attacker contract that re-enters `withdraw()` from its `receive()` fallback. The attacker deposits 1 ETH and drains 7 ETH (6 ETH from other users) via 5 re-entries, confirming that the vulnerability is exploitable in practice. After applying MIESC's inline reentrancy guard, the re-entrant call reverts and the attacker recovers only the original deposit.

Listing 3. Generated exploit test for reentrancy (simplified)

```
contract Attacker {
    Vault target;
    uint256 count;

    constructor(address _target) {
        target = Vault(_target);
    }

    function attack() external payable {
        target.deposit{value: 1 ether}();
        target.withdraw(1 ether);
    }

    receive() external payable {
        if (count < 5) {
            count++;
            target.withdraw(1 ether);
        }
    }
}

contract ReentrancyTest is Test {
    function test_drain() public {
        Vault vault = new Vault();
        vm.deal(address(vault), 6 ether);
        Attacker atk = new Attacker(
            address(vault));
        vm.deal(address(atk), 1 ether);
        vm.prank(address(atk));
        atk.attack();
        // Attacker drained 7 ETH (6 stolen)
        assertGt(address(atk).balance, 1 ether);
    }
}
```

The exploit-confirmation assertion (`assertGt(balance, 1 ether)`) passes on the vulnerable contract, confirming the drain. The fixed-version regression test inverts the expected outcome: it asserts that the re-entrant path is blocked or reverts after the guard is inserted. This two-artifact pattern—confirmation on the original, blocked-path verification on the patched version—is generated for each supported vulnerability category.

D. Stage 4: Formal Verification

The formal verification stage applies the Solidity compiler's built-in SMTChecker using the Constrained Horn Clauses (CHC) engine [16]. Unlike fuzzers that generate concrete test inputs, CHC-based verification checks encoded properties over symbolic execution paths via fixed-point computation over the contract's control-flow graph.

Methodology. For each finding, `miesc specs` generates one or more `assert()` statements encoding the property that should hold after patching. The SMTChecker is invoked with `--model-checker-engine chc` and `--model-checker-targets`

`assert,underflow,overflow`. We compare warnings between the original and patched contracts.

Results on illustrative contract. The vulnerable contract produces 3 SMTChecker warnings:

- **Underflow** at line 42 (`balances[msg.sender] -= amount`): reachable when `amount > balances[msg.sender]` after re-entry modifies the balance.
- **Assertion violation** on the generated `assert(address(this).balance >= sum_of_balances)` invariant: violated because re-entry drains more than the sender's legitimate balance.
- **Low-level call unchecked**: the `.call` return value discard path is reachable.

After patching, the first two warnings disappear entirely (the reentrancy guard makes the re-entry path unreachable), and the third is resolved by the `require(success)` wrapper. On this tractable example, CHC verification provides stronger evidence than the exploit test because it checks the encoded assertion over symbolic executions rather than over one concrete exploit trace.

Limitations. SMTChecker has known scalability limits: contracts with >10 external calls or complex inheritance hierarchies frequently hit the default 300-second timeout. On the SmartBugs corpus, only 23 of 143 contracts (16%) produce definitive CHC results within the timeout; the remainder either timeout or produce spurious warnings due to unresolved external calls. This limitation motivates the hybrid approach: exploit tests provide fast executable evidence, while formal verification adds stronger assurance only when the generated specification is tractable.

E. Stage 5: Compliance and Reporting

The compliance mapper maps each finding to applicable standards using a rule-based mapper with 12 standard profiles. On the illustrative contract, 4 of 11 findings map to MiCA Art. 11(1) (authorization and access control, ICT security policy), 3 to NIST CSF PR.AC-4 (access enforcement), and 2 to ISO 27001:2022 A.8.26 (application security). The report generator produces a 3,060-line HTML premium audit report with PoC templates, CVSS v3.1 estimates (base score computation from finding severity and attack vector), and a prioritized remediation roadmap.

F. End-to-End Pipeline

Table IV traces the full pipeline on a single illustrative contract. The complete pipeline—from scan to report—executes in under 10 seconds in local mode. For corpus-scale remediation results, see Tables I–II. Across the 143-contract corpus, the pipeline applies fixes to 99% of contracts, compiles 64% standalone, eliminates target-category HIGH/CRITICAL findings in 66%, and satisfies the bounded no-regression criterion for 65% of patched contracts.

TABLE IV
END-TO-END PIPELINE METRICS (ILLUSTRATIVE SINGLE CONTRACT)

Metric	Value
Total findings	11 (1C, 4H, 4M, 2L)
Patches applied	3 (+ 2 dedup)
HIGH vulns: original → after fix	3 → 0
Tests generated / compile / valid	6 / 6 / 6
Specs generated	1 (SMTChecker)
Standards mapped	MiCA (4 findings)
Report generated	HTML, 3060 lines
Pipeline time	< 10 seconds
API cost in local mode	$0

TABLE V
FEATURE COMPARISON WITH PRIOR REPAIR TOOLS. T/S/C = GENERATES EXPLOIT TESTS / FORMAL SPECS / COMPLIANCE MAPPING.

Tool	Yr	Vulns	Fix%	Comp.%	T	S/C
SCRepair [5]	'20	GP	—	—	Req[a]	×/×
sGuard [6]	'21	4	—	∼100	×	×/×
Elysium [7]	'22	7 SWC	+30[b]	—	×	×/×
ACFIX [8]	'24	1	94.9	—	×	×/×
SCPatcher [9]	'25	Gen.	81.5	91	×	×/×
MIESC	**'26**	**10 cats.**	**99**	**64**	✓	✓/✓

[a]Requires pre-existing test suite. [b]Relative to Shield.

G. Comparison with Prior Work

Table V compares MIESC against five prior automated repair tools. While SCPatcher reports a higher compilation rate (91% vs. 64%), its evaluation target and dependency assumptions differ from our standalone-compilation criterion, so the numbers are not directly interchangeable. ACFIX reports 94.9% fix rate but is limited to a single vulnerability class (RBAC). MIESC is the only evaluated tool that reports artifacts across the remediation lifecycle: patch generation, exploit testing, formal specifications, and compliance mapping.

H. Failure Analysis

Of the 141 contracts where a fix was applied, 51 (36%) fail standalone compilation. The per-contract compilation taxonomy classifies these failures as 42 undefined-symbol errors and 9 other compiler errors. The category-level artifact shows that unchecked low-level calls account for the largest share: 31 failures out of 51 applied fixes (61% category failure rate). Reentrancy has only one standalone failure (3%), while arithmetic has none.

This evidence supports a concrete root-cause direction for future work: dependency-aware compilation and symbol-resolution repair should be prioritized before semantic patch synthesis. The current taxonomy is still compiler-output based; it does not prove that every undefined symbol was introduced by the patcher, because some SmartBugs files already depend on missing libraries or legacy project context.

V. DISCUSSION

A. Limitations of Text-Level Patching

MIESC's patcher operates at the text level, not AST level. This design choice favors portability (works on any Solidity version without AST tooling) at the cost of precision. Specifically:

1) **No code restructuring**: the patcher cannot reorder statements to implement the Checks-Effects-Interactions (CEI) pattern—the gold standard for reentrancy prevention. Instead, it adds a mutex guard, which is a conservative mitigation but stylistically different from what a human auditor would write.
2) **Regex-based function matching**: unusual formatting (multi-line signatures, Unicode comments, preprocessor directives) can cause mismatched insertions. In our evaluation, compiler-output taxonomy still attributes 18 failures to miscellaneous compiler errors that require manual inspection.
3) **Single-contract scope**: cross-contract vulnerabilities (e.g., a proxy's `delegatecall` to a malicious implementation) cannot be patched because the fix requires coordinating changes across multiple files.

These limitations represent a fundamental tradeoff: text-level patching achieves 99% fix application rate across *all* Solidity versions (0.4–0.8) without requiring compilation, while AST-level approaches like sGuard achieve higher precision but are limited to specific compiler versions.

B. Compilation Rate Analysis

The 64% compilation rate deserves scrutiny. Under the same standalone criterion, all 141 original files that receive patches compile before modification, while 90/141 compile after patching. The remaining failures therefore measure patch-introduced or patch-exposed compilation problems, not merely missing benchmark dependencies. In the current run, 42/51 failures are undefined-symbol errors. Dependency-aware evaluation and symbol-resolution repair are required before claiming the effective compilation rate expected in a normal development repository.

C. Test Coverage and Correctness

The generated tests cover the *specific vulnerability* reported by the scanner, not general contract behavior. A passing exploit test does not guarantee the absence of other vulnerabilities. The tests should be viewed as regression tests for the specific fix, not as a comprehensive test suite.

Importantly, the exploit tests serve a dual role: (1) they provide executable evidence for the detection (if the test cannot trigger the vulnerability, the finding may be a false positive), and (2) they check the patched path exercised by the exploit (if the same exploit still succeeds after patching, the fix is incomplete). This bidirectional feedback loop is not reported by the repair tools in our comparison.

D. Cost and Scalability

The pipeline's execution cost depends on the LLM provider:

- **Local (Ollama)**: no API cost; runtime depends on the installed local model and hardware.
- **Frontier API**: provider-dependent cost; the dominant variables are context size, generated remediation text, and whether a single provider or ensemble is used.
- **Hybrid mode**: local-first execution can reserve frontier APIs for disputed HIGH/CRITICAL findings.

The practical implication is that remediation suggestions can be generated entirely offline, while teams that need higher semantic coverage can selectively pay for frontier review of the subset of findings that survive local triage.

E. Towards Continuous Security

The illustrative pipeline's sub-10-second execution time suggests a practical CI/CD integration pattern. A GitHub Action running `miesc scan --ci` on every pull request, combined with `miesc fix --dry-run` to suggest patches in PR comments, would provide continuous security feedback without blocking development velocity. This integration pattern—detection as a gate, remediation as a suggestion—preserves developer autonomy while making security findings visible earlier in the development process.

F. Threats to Validity

We identify four threats to the validity of our evaluation:

Internal validity. (1) *LLM non-determinism*: the detection stage can use frontier LLMs, whose outputs may vary across model versions and provider-side updates. The reproducible Paper 1 profile fixes the evaluated artifact and records layer selection, but future API runs may differ. (2) *Re-scan verification*: vulnerability elimination is measured by re-scanning the patched contract with the same pipeline family that detected the original vulnerability. A false negative on re-scan would inflate the elimination rate. We partially mitigate this by treating re-scan elimination as a bounded metric rather than as semantic proof and by separately reporting exploit tests and SMTChecker results where available.

External validity. (3) *Corpus selection bias*: SmartBugs-curated consists primarily of contracts from 2017–2019 with known vulnerabilities. These contracts are simpler (avg. 85 LoC) and more heavily studied than production DeFi protocols (avg. 500+ LoC with complex inheritance). Results on production code may differ, particularly for the compilation rate (which depends heavily on dependency availability). (4) *Vulnerability distribution*: the corpus over-represents reentrancy (31 contracts) and unchecked low-level calls (52 contracts) relative to their frequency in production code. Categories like oracle manipulation, MEV, and flash loan attacks—which dominate real-world exploits [17]—are absent from SmartBugs.

Construct validity. (5) *Compilation $\neq$ correctness*: a compiling patch is not necessarily semantically correct. The patch could introduce subtle behavioral changes (e.g., a reentrancy guard that blocks legitimate callback patterns). Our no-regression criterion measures only re-scan finding growth under the same pipeline, not preservation of all original functional behavior. (6) *Artifact granularity*: test generation, formal verification, and compliance reporting are evaluated end-to-end on an illustrative contract, while patch quality is measured corpus-wide. We therefore do not claim corpus-wide proof coverage for all generated artifacts.

Mitigation strategy. We address threats (3) and (4) by additionally reporting EVMBench results (120 business-logic vulnerabilities from real audit reports) in our companion paper [14], where MIESC achieves 92.5% recall on a corpus that includes oracle manipulation, flash loans, and governance attacks.

VI. Conclusion and Future Work

We have shown that the gap between vulnerability detection and actionable remediation artifacts can be substantially narrowed through automation. Evaluated on 143 contracts from the SmartBugs-curated corpus, MIESC achieves 99% fix application rate, 64% standalone compilation, 66% target-category vulnerability elimination by re-scan, and 65% pass rate under the bounded no-regression criterion. Compared to five prior repair tools (Table V), MIESC is the only system in our comparison that reports exploit tests, formal specifications, and compliance documentation alongside patches.

The key insight is that each stage *feeds the next*: detection informs patching, patching informs test generation, test generation checks the exploit path, and formal verification can add stronger evidence when the generated specification is tractable. This compositional architecture makes each intermediate artifact inspectable rather than forcing users to trust a single opaque remediation output.

Limitations. The 64% standalone compilation rate (Section V-F) reflects current patcher limitations under a strict standalone compilation criterion. The text-level patching approach cannot restructure code (e.g., apply CEI pattern) or handle cross-contract vulnerabilities. SMTChecker verification is tractable on only 16% of contracts within the default timeout, and corpus-wide semantic equivalence remains future work.

Future work. Three directions are planned: (1) *AST-level patching* via Slither's intermediate representation, enabling CEI reordering and cross-contract fixes; (2) *dependency-aware compilation* with automatic import resolution from npm/forge registries, which would eliminate the dominant compilation failure mode; and (3) *CI/CD integration* as a GitHub Action providing continuous security feedback on pull requests. We also plan to extend the evaluation to production DeFi protocols with complex inheritance and cross-contract interactions.

MIESC is available under AGPL-3.0 at https://github.com/fboiero/MIESC.

Acknowledgment

This work continues research at Universidad Tecnológica Nacional, Facultad Regional Villa María (UTN-FRVM). The authors thank the Ethereum security community and the developers of Foundry, Slither, and OpenZeppelin.

References

[1] J. Feist, G. Grieco, and A. Groce, "Slither: A static analysis framework for smart contracts," in *2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB)*. IEEE, 2019, pp. 8–15.

[2] G. Grieco, W. Song, A. Cygan, J. Feist, and A. Groce, "Echidna: Effective, usable, and fast fuzzing for smart contracts," in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2020, pp. 557–560.

[3] B. Mueller, "Smashing Ethereum smart contracts for fun and real profit," in *HITB Security Conference*, 2018.

[4] Y. Sun, D. Wu, Y. Xue, H. Liu, H. Wang, Z. Xu, X. Xie, and Y. Liu, "GPTScan: Detecting logic vulnerabilities in smart contracts by combining GPT with program analysis," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*. ACM, 2024, pp. 1–13.

[5] X. Yu, H. Zhao, B. Hou, Z. Ying, and B. Shi, "Smart contract repair," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 29, no. 4, pp. 1–32, 2020.

[6] T. D. Nguyen, L. H. Pham, J. Sun, Y. Lin, and Q. T. Minh, "sGuard: Towards fixing vulnerable smart contracts automatically," in *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2021, pp. 1349–1367.

[7] M. Rodler, W. Li, G. O. Karame, and L. Davi, "Elysium: Context-aware bytecode-level patching to automatically heal vulnerable smart contracts," in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2022, pp. 742–754.

[8] L. Zhang, K. Li, K. Sun, D. Wu, Y. Liu, H. Tian, and Y. Liu, "ACFIX: Guiding LLMs with mined common RBAC practices for context-aware repair of access control vulnerabilities in smart contracts," *IEEE Transactions on Software Engineering*, vol. 50, no. 7, pp. 1837–1853, 2024.

[9] S.-L. Li, Y.-T. Chen, and W.-C. Huang, "SCPatcher: Automated smart contract patching via RAG-enhanced LLM," *arXiv preprint arXiv:2604.00687*, 2025.

[10] Paradigm, "Foundry: Ethereum development toolkit," https://github.com/foundry-rs/foundry, 2025.

[11] V. Wüstholz and M. Christakis, "Harvey: A greybox fuzzer for smart contracts," https://arxiv.org/abs/2005.03350, 2020.

[12] A. Gupta, V. Gottimukkala, D. Robinson, Paradigm, and OpenAI, "EVMBench: Evaluating AI agents on smart contract security," *arXiv preprint arXiv:2603.04915*, 2026. [Online]. Available: https://github.com/paradigmxyz/evmbench

[13] M. di Angelo, T. Durieux, J. F. Ferreira, and G. Salzer, "SmartBugs 2.0: An execution framework for weakness detection in Ethereum smart contracts," in *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2023, pp. 2102–2105.

[14] F. Boiero and S. N. Mussetta, "MIESC: Multi-layer intelligent evaluation for smart contracts," *arXiv preprint*, 2026.

[15] J. F. Ferreira, P. Cruz, T. Durieux, and R. Abreu, "SmartBugs: A framework to analyze Solidity smart contracts," in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. ACM, 2020, pp. 1349–1352.

[16] a16z crypto, "Halmos: Symbolic testing for EVM smart contracts," https://github.com/a16z/halmos, 2023.

[17] N. Atzei, M. Bartoletti, and T. Cimoli, "A survey of attacks on Ethereum smart contracts (SoK)," *Proceedings of the 6th International Conference on Principles of Security and Trust (POST)*, pp. 164–186, 2017.